Persistent Runtime Brakes for Multi-Agent Coordination

Anonymous Author(s)
Affiliation
Address
email

Abstract

Agent frameworks can plan and protocols can move messages, but neither answers a prior question at the execution boundary: *may this coordination step run, and does a refusal still bind after the request ends?* We argue that many multi-agent failures in the wild are physical before they are semantic—stacked delegation, internal task bursts, and peer load export can exhaust a node while every hop remains a well-formed session. This short paper specifies a *consequence persistence layer* (CPL): an out-of-band control surface that adjudicates *before* irreversible intent or ingress, stores allow /delay /isolate outcomes on agent identity, and applies the same kernel to local planners and foreign peers. On an abstract 500-node load-exchange mesh, ungoverned coordination collapses to 0.4% mean survival, while pre-accumulation gating sustains 100% under the same seeds. Ablations show that a local circuit breaker is necessary and sufficient for *this* adversary; persistent reputation is motivated by open-mesh threats the harness does not stress. CPL is complementary to prompt-injection defenses and agent-control protocols: it governs optimization capacity, not message content.

1 Introduction

Deployed agents now decompose work, call tools, and talk to other agents [20, 19, 14, 2, 7]. Safety research has followed the new attack surface: indirect prompt injection, tool misuse, and computer-use compromise [9, 6, 1, 11]. A complementary failure mode is less discussed and already routine in multi-agent stacks. A planner fans out into $A \to B \to C$, sometimes closing a loop. Each hop can be a valid tool call or a polite protocol session. The *trajectory* still becomes physically unsustainable: recursion depth, context volume, concurrency, and neighbor load grow until the node saturates.

Per-request gateways forget. Token budgets account after commit. Framework `max_retries` counters reset when the session ends, and they cannot see a chain that crosses process or host boundaries [4]. In-process guardrails share the optimizer's address space and cannot be attested to a distrusting peer. Semantic protocols such as MCP and A2A [2, 7] decide *what* may be said; they are not brakes on *whether* the next optimization step may execute.

We call the missing object a **consequence persistence layer** (CPL). CPL sits at the execution boundary—the narrow interface where planning, delegation, and I/O meet compute, memory, and the wire—and enforces three contracts: (i) adjudicate *before* irreversible intent or ingress; (ii) bind the verdict to an agent or peer identity so it survives the request; (iii) use one authority for governing the local planner and admitting foreign traffic.

This is a short paper, not a claim that CPL replaces alignment, sandboxing, or injection defense. Our contributions are:

1. a layer diagnosis: planning and transport are populated; runtime coordination law is not;

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

2. a compact protocol—dual-path enforcement, a four-state machine, and a local viability score—that can wrap existing agent runtimes;
3. an existence experiment plus ablations that locate *which* mechanism carries survival under a reference load-export adversary, and which mechanisms remain conjectural for open meshes.

2 A consequence persistence layer

Dual path, one authority. A node faces two ingresses that must not split-brain. *Path A* (govern self) is a synchronous pre-flight probe from the local planner *before* it emits a tool call, delegation, or outbound message. *Path B* (govern neighbors) validates a physical *governance header before* any business payload is admitted. Both paths query the same **single execution authority** (SEA): one policy snapshot, one pressure monitor, one state machine. If Path A were strict and Path B lax, a locally throttled planner could still flood peers; the reverse lets inbound work overwhelm a node that has already stopped its own optimizer.

What is judged. SEA does not parse natural language, tool schemas, or session text. It evaluates a physical bundle: system pressure $e \in [0,1]$, planner recursion depth, a burst hint (estimated imminent fan-out), and a peer viability score $v \in [0,1]$. Reference pressure is a max-pool of normalized tool concurrency, memory, context volume, and burst:

$$e \;=\; \max\big(e_{\text{tool}},\, e_{\text{mem}},\, e_{\text{ctx}},\, e_{\text{burst}}\big). \tag{1}$$

Catastrophe is limited by the worst axis, not the average. Bands in the reference policy are $e_{\text{safe}} = 0.40$, $e_{\text{warn}} = 0.75$, and a hard limit $e^\star = 0.95$. Crossing $e^\star$ or a configured depth cap isolates immediately.

Consequences persist. Every probe returns one letter from a closed alphabet: ALLOW, DELAY(δ), or ISOLATE. There is no advisory mode. The letter is stored on identity, not on request id, and remains in force across scheduling epochs until a recovery rule fires. A four-state machine implements asymmetric hysteresis: PERMISSIVE $\leftrightarrow$ THROTTLED on pressure bands; ISOLATED after a signature failure, a malicious spike, or $v < v_{\text{crit}}$; PROBATIONARY only after a long dwell. Recovery is slow by construction (reference: 64 epochs isolated, 128 on probation, and $v \geq 0.8$ to return to full egress). Request completion does *not* clear isolation.

Open meshes: viability, not reputation theater. Inside an administrative boundary, Path A is the primary brake: the common failure is one's own planner. On an open peer mesh—the setting this workshop calls "in the wild"—strangers may connect, export load, and vanish. Path B therefore carries a governance header (depth, pressure hints, viability, optional collateral) as the first frame. The receiver *re-measures* pressure locally; self-reported safety is a claim, not a fact. For peer p at node n, **coordination viability** $v_n(p)$ is n's local estimate that p can keep coordinating without destabilizing n. It evolves as

$$v \;\leftarrow\; \text{clip}_{[0,1]}\big(\alpha v + \beta\,\tau - \gamma\, d - \delta\, e\big), \tag{2}$$

with inertia $\alpha = 0.95$, a small throughput reward $\beta = 0.05$, a destabilization penalty d, and locally remeasured e. Idle trust decays; recovery from $v_{\text{crit}} = 0.3$ is logarithmic. Below v_{crit} the peer is coordination-bankrupt: admission fails regardless of session politeness. First-seen peers in the open profile additionally pay a stranger tax (minimum collateral) so hit-and-run flooding is not free.

Placement. CPL is out-of-band: it must not depend on the application protocol cooperating. A reference implementation exposes pre-flight as local IPC and wraps planner nodes in graph-based agent frameworks so a blocked step can halt or annotate state instead of throwing the process into an ungoverned retry storm. Enterprise and open-mesh deployments share the same kernel; only the trust posture on Path B changes.

3 Existence evidence, not a field trial

We do *not* evaluate prompt injection, computer use, or task success. The question is narrower and prior: under random pairwise load exchange, does pre-accumulation gating sustain a mesh that otherwise dies?

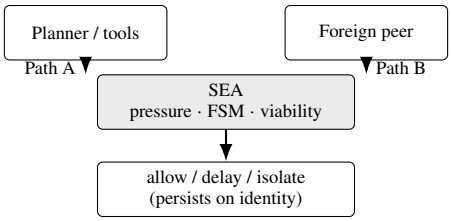


Figure 1: CPL in one picture. Path A brakes the local optimizer before intent leaves; Path B brakes neighbors before payload admission. Both share one authority and one persistent consequence.

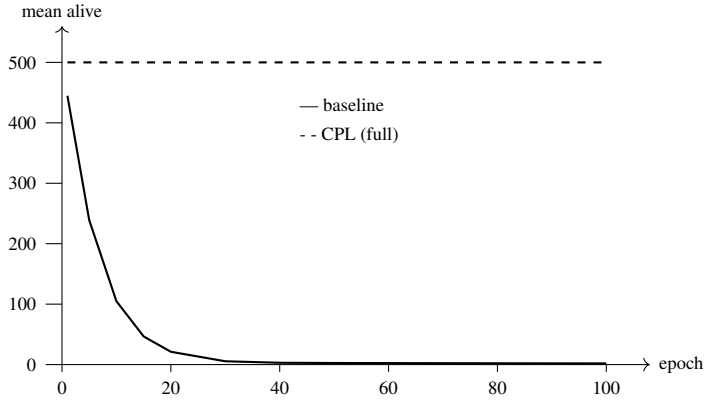


Figure 2: Mean alive nodes over $1{,}000$ paired runs ($N = 500$, 5% malicious). Ungoverned exchange collapses; pre-accumulation gating does not.

Harness. $N = 500$ nodes, 5% designated malicious exporters, 100 epochs, $1{,}000$ paired Monte Carlo runs, fan-out 3. Each epoch, surviving nodes cool by $\times 0.5$, then each sends three requests. Benign request load is 0.1; malicious load is 0.8. A node dies irreversibly if load ≥ 1. Baseline accepts every request. CPL mode applies three pre-accumulation rules aligned with Section 2: refuse if the target is below $v_{\text{crit}} = 0.3$; refuse if $\text{load} + \text{request} > 0.95$ and penalize the source (asymmetric: -0.5 if malicious, -0.1 otherwise); penalize large bursts ($\text{request} > 0.5$) even when accepted. The harness collapses pre-flight, headers, and gossip into scalar load and v. It preserves control-law *ordering* (reject before accumulate), not wire fidelity.

Reference gap. At $T = 100$, baseline mean survival is $1.88/500$ nodes (0.38%). CPL mode keeps $500/500$ under identical seeds (Figure 2). Baseline dies on a short horizon: mean alive is already $\approx 21\%$ by epoch 10 and $< 1\%$ by epoch 31. The gap is a phase transition—positive feedback without a brake versus negative feedback with one—not a small quality improvement.

What actually carries the result. Table 1 removes one gate at a time (300 runs). A local circuit breaker is *necessary and sufficient* against this exporter: breaker-only matches full CPL at 100%, while viability-only matches baseline. Burst tax and the viability floor do not move survival when the breaker is present, because a node that refuses any request that would cross 0.95 never reaches fatal load. That is an invariant of the model, not a miracle of reputation. We treat this as a feature of the claims boundary. Persistent viability and stranger tax are designed for hit-and-run, forwarding contagion, and first-seen peers—threats this random-exchange harness does not instantiate. A paper that sold 100% vs. 0.4% as proof that the full trust stack is necessary would be overclaiming.

Sensitivity. Sweeping the malicious fraction from 1% to 50% does not dent CPL-mode survival: the breaker still refuses lethal increments. Baseline stays near zero once exporters exist. Reducing malicious per-request load to 0.45, 0.20, or 0.12 (below the burst-tax threshold) still collapses baseline (0.6%, 3.1%, 7.8%) while CPL remains at 100%. Fragmenting load evades the burst tax; it does not evade pre-accumulation refusal once many small hits would saturate a target. An adaptive

Table 1: Mean survival (%) at $T = 100$. The breaker, not viability, carries this adversary.

Malicious %	Full	No v floor	No breaker	No burst tax	Breaker only	v only / base
5	100	100	4.8	100	100	0.3 / 0.3
20	100	100	15.7	100	100	0.2 / 0.2

adversary that *also* lies about headers, shards work across fresh identities, or attacks the attestation channel is out of scope for the harness and listed as future work.

Limitations. The experiment is an abstract mesh, not Claude Code, OpenClaw, or a production MCP deployment. It does not prove liveness, wire-faithful sidecar replay, gossip dynamics, or optimal coefficients. It does show that ungoverned load exchange can fail sharply, and that reject-before-accumulate is enough to prevent that failure under the reference exporter.

4 Related work

Agent frameworks decompose and converse [20, 19, 10, 15, 14, 5, 16, 18]. They optimize task completion. Failure taxonomies now document coordination breakdowns [4], but the prescribed fixes remain prompts, retries, and better roles—not a persistent runtime brake.

Agent safety evaluations measure harmfulness, injection, and tool abuse [6, 1, 17, 9]. These are the right tests for semantic and capability risk. They do not ask whether a well-formed multi-agent graph can still saturate its host or its neighbors.

Control and oversight. AI control [8], debate [12], constitutional feedback [3], scalable oversight [13], and representation-level circuit breakers [21] constrain *what the model is trying to do*. CPL constrains *how much optimization is allowed to happen*. A monitored but unbraked fleet can still loop itself into unavailability.

Agent protocols. MCP and A2A [2, 7] plus recent MCP security analyses [11] secure tools, schemas, and sessions. CPL is intentionally off that path: a semantically valid MCP call can still be a physically illegal coordination step. The two layers compose—CPL first, then the application protocol.

5 Discussion

Agents in the wild fail in more than one language. One language is content: injections, jailbreaks, malicious tools. The other is physics: stacked optimizers that never emit a "bad" message. Most current defenses speak the first language. CPL is a proposal for the second.

What we claim: runtime coordination needs a persistent, pre-intent, out-of-band layer; a minimal reject-before-accumulate law prevents cascade death in a reference load-export model; viability and stranger tax are the open-mesh remainder, not yet load-bearing in that model. What we do not claim: field-validated safety, formal verification, or substitution for injection defense, sandboxing, or human oversight.

The next measurements are the ones this workshop is built for: pre-flight hit rates on a real planner graph, CPL in front of MCP ingress, and an exporter with an identity budget. Until those traces exist, CPL is a specified brake with existence evidence—not a completed in-the-wild safety result.

References

[1] Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, et al. AgentHarm: A benchmark for measuring harmfulness of LLM agents. *arXiv preprint arXiv:2410.09024*, 2024.

[2] Anthropic. Model context protocol. `https://modelcontextprotocol.io`, 2024.

[3] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

[4] Mert Cemri, Melissa Z Pan, Shuyi Yang, Lakshya A Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, et al. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*, 2025.

[5] CrewAI. CrewAI. `https://www.crewai.com`, 2025.

[6] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems*, 2024.

[7] Google. Agent2agent protocol (A2A). `https://a2a-protocol.org`, 2025.

[8] Ryan Greenblatt, Buck Shlegeris, Kshitij Sachan, and Fabien Roger. AI control: Improving safety despite intentional subversion. In *International Conference on Learning Representations*, 2024.

[9] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, 2023.

[10] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023.

[11] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. Model context protocol (MCP): Landscape, security threats, and future research directions. *arXiv preprint arXiv:2503.23278*, 2025.

[12] Geoffrey Irving, Paul Christiano, and Dario Amodei. AI safety via debate. In *NeurIPS Workshop on Ethical, Social and Governance Issues in AI*, 2018.

[13] Zachary Kenton, Noah Siegel, János Kramár, Jonah Brown-Cohen, Samuel Albanie, Jannis Bulian, Rishabh Agarwal, David Lindner, Yunhao Tang, Noah Goodman, et al. On scalable oversight with weak LLMs judging strong LLMs. *arXiv preprint arXiv:2407.04622*, 2024.

[14] LangChain. LangGraph. `https://langchain-ai.github.io/langgraph/`, 2025.

[15] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Alan Ghanem. CAMEL: Communicative agents for "mind" exploration of large language model society. In *Advances in Neural Information Processing Systems*, 2023.

[16] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023.

[17] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. In *International Conference on Learning Representations*, 2024.

[18] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 2023.

[19] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

[20] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[21] Andy Zou, Long Phan, Justin Wang, Derek Duenas, Maxwell Lin, Maksym Andriushchenko, Rowan Wang, Zico Kolter, Matt Fredrikson, and Nicholas Carlini. Improving alignment and robustness with circuit breakers. In *Advances in Neural Information Processing Systems*, 2024.

A Supplementary material

This appendix does not count toward the 4-page short-paper limit. It records constants, additional numbers, and an implementation sketch without identifying links.

A.1 Reference constants

Table 2: Harness and kernel constants used in Section 3 and the reference control plane.

Symbol / name	Value
N, epochs, runs (main curve)	500, 100, 1,000
Runs (sweeps / ablations)	300
Fan-out, cool-down	3 requests / node / epoch, $\times 0.5$
Benign / malicious request load	0.1 / 0.8
Fatal load, breaker threshold $e^\star$	1.0, 0.95
v_{crit}, burst-tax threshold	0.3, 0.5
Breaker penalties (malicious / benign)	-0.5 / -0.1
Burst-tax penalty	-0.3
Eq. (2): α, β	0.95, 0.05
Idle decay λ, recovery κ	0.01 / epoch, 0.02
Isolation / probation dwell	64 / 128 epochs
Permissive re-entry floor	$v \geq 0.8$
Pressure bands e_{safe}, e_{warn}	0.40, 0.75

A.2 Malicious-rate and fragmented-load sweeps

Table 3: Mean survival (%) at $T = 100$ vs. malicious fraction (300 runs).

Mode	1%	5%	10%	15%	20%	30%	40%	50%
Baseline	2.3	0.3	0.3	0.3	0.2	0.2	0.2	0.2
CPL (full)	100	100	100	100	100	100	100	100

Table 4: Mean survival (%) when malicious per-request load is reduced (5% malicious, 300 runs).

Malicious load	0.80	0.45	0.20	0.12
Baseline	0.3	0.6	3.1	7.8
CPL (full)	100	100	100	100

A.3 Why viability looks idle in the harness

Viability v decreases only when the breaker or burst tax fires. A viability-only policy therefore never observes $v < v_{\mathrm{crit}}$ unless some other rule first writes a penalty. That is why "v only" matches baseline in Table 1. In the full protocol, v is written by Path B attestation failures, stranger-tax refusals, and gossiped isolation warnings—none of which the abstract harness models. The correct reading is: *local physical refusal is the survival mechanism against dumb exporters; viability is a mesh-memory mechanism awaiting a contagion / hit-and-run eval.*

A.4 Implementation sketch (anonymized)

A conforming Path A runtime:

1. reports recursion depth and context bytes to the pressure monitor;
2. issues a blocking pre-flight RPC with a burst hint;
3. proceeds only on ALLOW, sleeps δ on DELAY, and must not emit I/O on ISOLATE.

A conforming Path B hop sends a length-prefixed governance header as frame 1 and treats the header as a claim to be re-measured. Enterprise (closed) and open-exchange profiles share entropy, depth, and FSM law; the open profile adds collateral and header attestation. Code and deployment names are withheld for double-blind review and can be restored in the camera-ready version.